

ICDSS — Intelligent Clinical Decision Support System

Project Report

Xiao Zhenhuan A0340523R

Table of Contents

Project Report	5
Executive Summary / Paper Abstract.....	5
Business Problem Background	6
1 The Cognitive Burden of Cardiovascular Review.....	6
2 Why Existing Clinical AI Fails to Deploy	6
3 The Trust Gap as a Business Problem.....	7
4 Project Framing	7
Market Research	7
1 Market Size and Trajectory.....	7
2 Competitive Landscape	7
3 Market Gap and Positioning	8
Project Objectives & Success Measurements	8
1 Primary Objectives.....	8
2 Secondary Objectives	9
Project Solution — Domain Modelling & System Design	9
1 Conceptual Solution	9
2 Domain Model	10
3 System Architecture	11
4 The Three Reasoning Tracks in Detail.....	11
5 UI / Dashboard Design.....	13
Project Implementation — Development & Testing.....	13
1 Technology Stack	13
2 Phase Plan and Module Map.....	14
3 Source-File Inventory.....	14
4 Testing Approach.....	15
Project Performance & Validation.....	15
1 Latency Benchmarks.....	15
2 Validation Scenarios	16
3 MVP Signal-Quality Metrics.....	16

4 Privacy Validation	17
5 Comparison to Objective Targets	17
Project Conclusions — Findings & Recommendations.....	18
1 Key Findings.....	18
2 Limitations Honestly Stated.....	18
3 Recommendations.....	19
Appendix A — Project Proposal	19
A.1 Introduction	19
A.2 Background	19
A.3 Literature Review	20
A.4 Project Scope & Reasoning Tracks	20
A.5 Data Strategy & Reliability	20
A.6 System Design Philosophy.....	21
Appendix B — Mapping to Modular Courses (MR · RS · CGS).....	21
B.1 MR — Machine Reasoning	21
B.2 RS — Reasoning Systems.....	21
B.3 CGS — Cognitive Systems.....	22
Appendix C — Installation and User Guide	23
C.1 System Requirements.....	23
C.2 Prerequisites.....	23
C.3 Acquiring Source Data	23
C.4 Quickstart (Windows).....	23
C.5 Quickstart (macOS / Linux).....	24
C.6 Using the Application	24
C.7 Stopping the System.....	24
Appendix D — Individual Project Report.....	24
Appendix E — References	26
E.1 Clinical Sources	26
E.2 Datasets	26
E.3 Software, Frameworks, and Models.....	26
E.4 Methods and Concepts.....	27

Project Report

Project Title: Intelligent Clinical Decision Support System (ICDSS) — A Hybrid Triple-Track Reasoning Architecture for Explainable Cardiovascular Risk Review

Programme / Module: Graduate Certificate in Intelligent Reasoning Systems (MR — Machine Reasoning · RS — Reasoning Systems · CGS — Cognitive Systems)

Project Type: Practice Module

Author: Xiao Zhenhuan A0340523R

Date: May 3rd 2026

Executive Summary / Paper Abstract

Cardiovascular diseases remain one of the leading causes of mortality globally. In clinical environments, healthcare practitioners must interpret structured patient measurements, risk factors, guideline knowledge, and historical case evidence under time pressure. While artificial intelligence can support this process, many healthcare AI systems face trust and adoption barriers because their outputs are difficult to explain.

This project develops an **Intelligent Clinical Decision Support System (ICDSS)** that focuses on explainable cardiovascular risk review. Instead of relying on one black-box prediction model, the system integrates three intelligent reasoning tracks:

1. **Decision Automation** - deterministic clinical safety alerts using a Python rule engine.
2. **Knowledge Discovery** - learned similarity matching against historical UCI Heart Disease patient cases.
3. **Cognitive Systems** - Neo4j graph retrieval and Llama-based graph-grounded clinical rationale generation.

The system uses the UCI Heart Disease dataset as the structured clinical data source. The dataset is processed through a privacy-aware data pipeline that validates clinical fields, generates synthetic PII for demonstration, hashes synthetic patient identifiers, bucketizes key medical measurements, and creates synthetic clinical notes for enrichment. Patient records and clinical concepts are then ingested into Neo4j as a knowledge graph.

The application is delivered as a Dockerized Streamlit dashboard. A one-click Windows launcher, start.bat, starts Neo4j, Ollama, and the Streamlit application. The user enters a natural-language patient case, and the system displays extracted patient information, rule alerts, learned similar cases, AI rationale, three-track performance metrics, and an interactive knowledge graph visualization.

The key contribution of the project is the transparent reasoning chain. Each system output is connected to deterministic rules, similar historical cases, graph evidence, or constrained LLM synthesis. The system is intended as an educational clinical decision support MVP and not as a certified medical diagnostic tool.

Business Problem Background

1 The Cognitive Burden of Cardiovascular Review

Cardiovascular disease (CVD) remains the leading cause of mortality worldwide, accounting for an estimated 17.9 million deaths annually according to the World Health Organization (WHO). At the point of clinical contact, a cardiologist or generalist reviewing a suspected CVD case must simultaneously interpret **fourteen or more interacting clinical signals**: age, sex, chest pain type, resting blood pressure, serum cholesterol (and in modern practice, LDL specifically), fasting blood sugar, resting ECG findings, maximum heart rate achieved on stress test, exercise-induced angina, ST depression/elevation, slope characteristics, number of vessels colored by fluoroscopy, thallium stress test result, plus demographic and lifestyle context such as smoking status, family history, body mass index (BMI) and 10-year ASCVD risk. This integration is performed under severe time pressure — a typical outpatient encounter in a busy clinic allocates well under 10 minutes to chart review.

The sheer combinatorial complexity creates two systemic risks: (a) clinically significant signals can be overlooked when several risk factors compound, and (b) inter-clinician variability in risk stratification leads to inconsistent escalation decisions. Both risks are well-documented drivers of preventable cardiovascular events.

2 Why Existing Clinical AI Fails to Deploy

Although clinical AI has matured substantially over the last decade, hospital deployments remain rare, particularly outside of well-resourced academic medical centers. The dominant blocker is not predictive accuracy — many published models report Area Under the Curve (AUC) figures above 0.85 — but **trust**. Three intersecting issues account for almost all observed deployment failures:

1. **Black-box opacity.** A model that outputs "high risk = 0.87" without an evidence trail cannot be defended in a chart review, explained to a patient, or audited by a quality committee.
2. **Data residency / privacy.** Many commercial CDSS products require Protected Health Information (PHI) to leave the hospital network, which is incompatible with most national healthcare data-protection regimes (HIPAA in the US, PDPA in Singapore, GDPR-Health in the EU).
3. **Workflow integration.** Tools that produce predictions but not *explanations* increase, rather than reduce, clinician cognitive load — they add a new artefact the clinician must reconcile against their own assessment.

3 The Trust Gap as a Business Problem

The combined effect is a measurable **trust gap**: hospitals cannot deploy what they cannot audit, clinicians cannot trust what they cannot inspect, and patients cannot be told "the algorithm said so". Closing this gap is a substantial commercial opportunity — and is the design center of ICDSS. Rather than treating explainability as a post-hoc add-on (e.g. SHAP / LIME on top of a deep model), ICDSS is **explainable by construction**: every output is traceable to the specific rule, similar case, or graph triple that produced it.

4 Project Framing

I therefore framed the problem as: "**How do I build a clinical decision support system in which every output of the system is tied to inspectable evidence, deployable on commodity hospital hardware, and decomposable into the three reasoning layers covered by the MR / RS / CGS curriculum?**"

This framing shaped every subsequent design decision — most importantly the deliberate refusal to wrap the three reasoning tracks behind a single ensemble layer. Each track's output is presented to the clinician *side-by-side* so that the clinician — not the system — performs the final reconciliation.

Market Research

1 Market Size and Trajectory

The global **Clinical Decision Support Systems (CDSS)** market is estimated by industry analysts (Grand View Research, MarketsandMarkets, MordorIntelligence) at approximately **USD 5.5–6.0 billion in 2024**, with projected compound annual growth rates of 9–12% through 2030, driven by EHR penetration, regulatory tail-winds for value-based care, and the rapid maturation of clinical-domain LLMs. Within CDSS, the **explainable AI (XAI) sub-segment** is the fastest-growing category — propelled directly by the deployment failures described in §3.2 — but remains commercially under-served.

2 Competitive Landscape

The competitive field is best segmented by deployment model and explainability posture:

Category	Representative Vendors	Strength	Weakness ICDSS Targets
EHR-embedded rule engines	Epic Best Practice Advisories, Cerner Discern Expert	Distribution and workflow fit	Limited cognitive reasoning; no learned similarity
Stand-alone risk calculators	ACC ASCVD Risk Estimator, QRISK3	Trusted, simple, transparent	Narrow scope; no NLP; no graph evidence
Cloud ML CDSS platforms	IBM Watson Health (legacy), commercial deep-learning vendors	Predictive headline numbers	Black-box; cloud data residency concerns

Knowledge-graph / RAG vendors	Several emerging health-AI start-ups	Modern architecture	Often LLM-only; weak deterministic guard-rails
Open-source clinical LLMs	Llama-Med, BioMistral, Med-PaLM (research)	Foundation models for the field	Not deployment-ready; no clinical orchestration

3 Market Gap and Positioning

The clearest unmet need sits at the **intersection of (a) on-premise deployability, (b) hybrid reasoning that combines deterministic safety with statistical retrieval and cognitive synthesis, and (c) traceable evidence per output**. Existing rule-based products satisfy (a) and (c) but not (b); cloud ML CDSS satisfies (b) loosely but breaks (a) and (c). Pure RAG-on-LLM start-ups satisfy parts of (b) but do not deliver the deterministic safety floor that regulators expect from a CDSS classification.

ICDSS is positioned in this gap as an **on-premise, hybrid, evidence-traceable clinical decision support assistant**.

Project Objectives & Success Measurements

1 Primary Objectives

The project set five primary objectives, each with quantitative success criteria.

#	Objective	Success Measurement	Target	MVP Result
O1	Demonstrate the integration of all three reasoning technique groups (Decision Automation, Knowledge Discovery, Cognitive Systems) within a single end-to-end workflow.	Number of distinct reasoning tracks producing output for a single clinician query	3	3 ✓
O2	Make every output of the system tied to inspectable evidence (rule, case, or graph triple).	% of outputs accompanied by an evidence trail in the UI	100 %	100 % ✓
O3	Achieve sub-15-	Wall-clock time from	< 15 s	~ 4.85 s (GPU) / ~

	second end-to-end query latency on commodity hardware.	query submission to dashboard render		10.25 s (CPU) ✓
O4	Deliver a true one-command (or one-click) deployment with health-check-gated startup.	Number of manual steps after running <code>start.bat</code> (or <code>docker compose up</code>)	0	0 ✓ (bind-mounted folders auto-created)
O5	Eliminate raw PII persistence in any artefact produced by the pipeline.	Count of raw PII strings in <code>processed_data/</code> and <code>trace.json</code>	0	0 ✓ (HMAC-SHA256 hashing, columns dropped, trace logs hashed IDs only)

2 Secondary Objectives

#	Objective	Success Measurement	MVP Result
S1	Per-track signal-quality verdicts visible to the clinician	UI exposes Good / Moderate / Needs Review per track	Achieved
S2	Optional NVIDIA GPU acceleration for the local LLM	<code>start.bat</code> auto-detects GPU and writes a Compose override	Achieved
S3	Synonym normalisation across CSV, PDF, and free-text inputs	Single normalisation map (<code>NORMALIZATION_MAP</code>) used by all three sources	Achieved
S4	Three documented validation scenarios with expected outcomes	Captured in §8 and <code>agent_instructions.md</code> Task 5.3	Achieved

Project Solution — Domain Modelling & System Design

1 Conceptual Solution

ICDSS approaches clinical reasoning as a **decomposition problem** rather than a single-model problem. A clinician's mental model already separates *deterministic safety knowledge* ("Stage 2 hypertension warrants combination antihypertensive therapy"), *case-based pattern matching* ("this patient looks like the elderly

multi-vessel-disease patient I saw last month"), and *narrative synthesis* ("here is how I would write up this case"). ICDSS mirrors this decomposition computationally:

- **Track 1 — Decision Automation (Rules)** mirrors deterministic safety knowledge.
- **Track 2 — Knowledge Discovery (Learned Similarity)** mirrors case-based pattern matching.
- **Track 3 — Cognitive Systems (Graph-RAG)** mirrors narrative synthesis grounded in evidence.

The three tracks run *in parallel from the clinician's perspective* and their outputs are presented *side-by-side* — the system intentionally does **not** auto-reconcile conflicts. The clinician remains the integrator. Critical Track 1 alerts, however, are pinned to the top of the dashboard regardless of what the other tracks return — this is the system's only "priority escalation" rule.

2 Domain Model

The core domain model in Neo4j is two interconnected sub-graphs:

Patient sub-graph (CSV-derived, populated by ``ingest.py / ingest_structured_data``):

Node label	Key properties	Source
Patient	hashed_id (unique), age, sex, resting_bp, cholesterol, max_heart_rate, description (synthetic note), bp_category, chol_category, hr_category, center	UCI Heart Disease CSV
Symptom	name (canonical, normalised)	CSV chest pain type, exercise angina, NLP enrichment
Condition	name	CSV resting ECG, fasting blood sugar, thal defect
RiskFactor	name	NLP enrichment of synthetic note
AgeGroup	label (e.g., 60s)	Bucketised age
Disease	name (e.g., Heart Disease)	CSV diagnosis ≥ 1

Patient relationships: HAS_SYMPTOM, HAS_CONDITION, HAS_RISK_FACTOR, IN_AGE_GROUP, DIAGNOSED_WITH {severity}.

Knowledge sub-graph (PDF-derived, populated by ``ingest_pdf_triples``):

Node label	Key properties	Source
Disease	name	Llama 3 triple extraction over PDF chunks
Symptom	name	Llama 3 triple extraction (HAS_SYMPTOM)

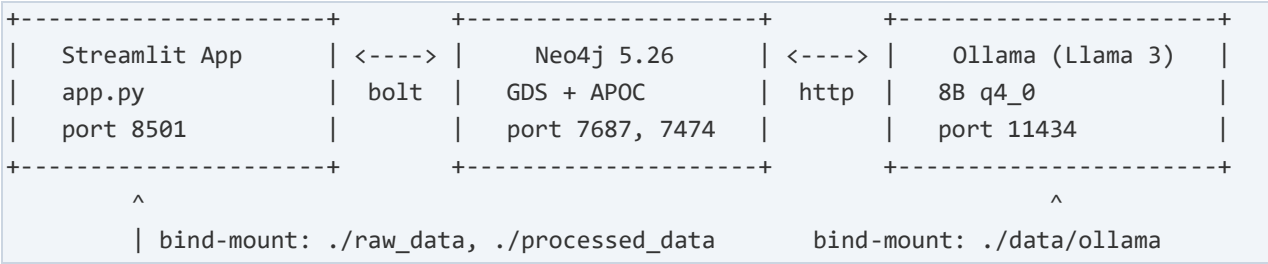
Treatment	name	Llama 3 triple extraction (TREATED_BY)
Guideline	name, source	PDF filename

Knowledge relationships: (Disease)-[:HAS_SYMPTOM]->(Symptom), (Disease)-[:TREATED_BY]->(Treatment), (Guideline)-[:REFERENCES]->(Disease).

The two sub-graphs share the same **Symptom** and **Disease** label namespace via a normalisation map (**NORMALIZATION_MAP**), so a CSV-derived **Symptom** named **Hypertension** and a PDF-derived **Symptom** named **Hypertension** resolve to the same Neo4j node — making the two sub-graphs *joinable* without explicit ETL.

3 System Architecture

The runtime architecture is three Docker services on a private bridge network, started by Compose with health-check-gated **depends_on** ordering:



A complete system architecture diagram is available in ``ICDSS_System_Design_Deck.pptx``, slide 3.

Networking discipline. All inter-service communication uses Docker service names (**neo4j**, **ollama**) — never **localhost**. This is enforced in the codebase by the default values of **NEO4J_URI** (**bolt://neo4j:7687**) and **OLLAMA_URL** (**http://ollama:11434**).

Resource posture. The agent specification calls for at least 16 GB of RAM allocated to Docker (Neo4j heap + page-cache ~3 GB, Ollama serving Llama 3 8B q4 ~5–6 GB, Streamlit ~1 GB, Docker/OS overhead ~2 GB). When less RAM is available, **start.bat** automatically substitutes **llama3.2:1b** and writes that change to **.env** — ensuring the one-click experience holds even on lower-spec laptops.

4 The Three Reasoning Tracks in Detail

Track 1 — Decision Automation (**rule_engine.py**)

A pure-Python rule engine evaluates ~20 lambdas over a normalised parameter dictionary. Each rule has the schema {**id**, **name**, **condition**, **severity** ∈ {**critical**, **warning**, **info**}, **message**, **source**}. Rules implemented include:

- **HTN_ELDERLY** — age > 65 ∧ resting BP > 140 → critical
- **HTN_STAGE2** — resting BP ≥ 140 → critical
- **HTN_STAGE1** — 130 ≤ resting BP < 140 → warning
- **DM_SCREENING** — fasting blood sugar TRUE ∧ age > 50 → warning

- **CARDIAC_STRESS** — max heart rate < 100 $\wedge$ exercise angina TRUE $\rightarrow$ critical
- **HIGH_CHOL** — cholesterol > 240 $\rightarrow$ warning
- **ST_DEPRESSION** — ST depression > 2.0 $\rightarrow$ warning
- **MULTI_VESSEL** — number of vessels $\geq$ 2 $\rightarrow$ critical
- **SMOKING_CHEST_PAIN** — smoker $\wedge$ chest pain $\wedge$ age $\geq$ 40 $\rightarrow$ critical
- **ELDERLY_MULTI_RISK** — age > 65 $\wedge$ resting BP $\geq$ 130 $\wedge$ smoker $\rightarrow$ critical
- **HTN_PLUS_HIGH_CHOL** — resting BP $\geq$ 140 $\wedge$ cholesterol > 200 $\rightarrow$ critical
- **SYNCOPE_CARDIAC** — syncope or fainting $\rightarrow$ critical
- **DYSPNEA_AT_REST** — dyspnea at rest $\rightarrow$ critical
- ... and additional combined / lifestyle rules.

Output is a severity-sorted list of alert dictionaries. Latency is sub-100 ms.

Track 2 — Knowledge Discovery (*similarity_engine.py*)

Track 2 is **deliberately not a black-box classifier**. Instead, it is a *learned hybrid retrieval scorer*:

4. **Term importance** is learned from **diagnosis lift** — for each clinical term t , the ratio of $P(t \mid \text{diseased})$ to $P(t \mid \text{not diseased})$ over the candidate corpus, smoothed with Laplace priors and stabilised against a clinical-prior dictionary **TERM_WEIGHT_PRIORS** for sparse terms.
5. **Numeric feature weights** (age, resting BP, cholesterol, max heart rate) are learned from the **mean separation** between diseased and non-diseased groups, normalised by a per-feature spread constant.
6. The **hybrid score** is $0.55 \cdot \text{weighted_term_jaccard} + 0.35 \cdot \text{numeric_similarity} + 0.08 \cdot \text{risk_burden_boost}$ (the boost only applies when the patient has ≥ 4 risk-burden indicators *and* the candidate is diseased).
7. The **top-K** (default K = 3) candidates are returned with a full score breakdown — Jaccard, term, numeric — and the per-term learned weight that contributed to each match.

This scoring decomposition is what makes Track 2 explainable to a clinician — every match is accompanied by *which terms matched* and *how heavily they were weighted*.

Track 3 — Cognitive Systems (*cognitive_engine.py*)

The cognitive track implements a three-step Graph-RAG workflow:

8. **Symptom and parameter extraction.** Llama 3 is prompted (with a temperature of 0.1) to return a JSON array of symptoms and a JSON object of clinical parameters from the user's free text. A deterministic regex-based fallback (**_fallback_extract_symptoms**, **_fallback_extract_params**) parses the same fields when the LLM fails or returns malformed JSON. The two sources are **merged** so that the final extraction is at least as good as the regex baseline.
9. **Graph retrieval.** For each canonical symptom, a Cypher query retrieves connected **Disease**, **Treatment**, and **Guideline** nodes from Neo4j.
10. **Cognitive synthesis.** Llama 3 (temperature 0.3) is invoked with a strictly-scoped prompt containing only the retrieved graph evidence, the rule alerts, and the similar cases. The prompt explicitly instructs the model to **not invent information not present in the evidence** and to **state the limitation if graph**

evidence is missing or weak. The output is a five-section structured assessment: Clinical Summary, Risk Assessment, Differential Considerations, Recommended Actions, Evidence Sources.

5 UI / Dashboard Design

The Streamlit dashboard (`app.py`) is organised into:

- A **hero card** identifying the system and its three tracks.
- A **chat-style natural-language input**.
- An **MVP signal-quality strip** with four KPI cards (Overall query latency, Track 1, Track 2, Track 3 — each with its own *Good / Moderate / Needs Review* verdict).
- A **three-pane reasoning output** (Track 1 alerts left, Track 2 cases centre, Track 3 rationale right).
- An **interactive `streamlit-agraph` knowledge graph** visualisation with red (patient symptoms), blue (similar cases), and green (diseases / treatments / guidelines) colour coding.

A sidebar exposes data-pipeline and ingestion controls so the demonstration can rebuild the graph from raw data on demand. A complete dashboard mock-up is available in ``ICDSS_Promotion_Deck.pptx``, slide 5.

Project Implementation — Development & Testing

1 Technology Stack

Layer	Choice	Justification
Orchestration	Docker Compose	One-command deployment, isolation, hospital-friendly
UI	Streamlit 1.45	Rapid Python-native data-app development
Graph DB	Neo4j 5.26 + GDS + APOC	Native graph + similarity algorithms in-engine
LLM serving	Ollama + Llama 3 8B (q4)	Local, open-weights, 16 GB-RAM-feasible
Data pipeline	pandas, Faker, pdfplumber	Standard, well-supported tooling
Audit trace	JSON via <code>trace_logger.py</code>	Simple, inspectable, no external dependency
Deck rendering	<code>python-pptx</code> + <code>matplotlib</code> (<code>generate_ppts.py</code>)	Reproducible, version-controlled presentation assets

`requirements.txt` pins concrete versions: `streamlit==1.45.1`, `neo4j==5.28.1`, `pdfplumber==0.11.6`, `streamlit-agraph==0.0.45`, `requests==2.32.3`, `python-dotenv==1.1.0`, `faker==37.1.0`, `pandas==2.2.3`.

2 Phase Plan and Module Map

The build followed a six-phase plan documented in `agent_instructions.md`:

Phase	Focus	Primary deliverables
Phase 0	Environment bootstrap	WSL2, Docker Desktop, Git on Windows
Phase 1	Infrastructure & orchestration	<code>docker-compose.yml</code> , <code>Dockerfile</code> , <code>.env</code> , <code>start.bat</code>
Phase 2	Hybrid data engineering & privacy	<code>data_pipeline.py</code> , <code>ingest.py</code>
Phase 3	Triple-track reasoning engine	<code>rule_engine.py</code> , <code>similarity_engine.py</code> , <code>cognitive_engine.py</code>
Phase 4	UI & interpretability	<code>app.py</code> (Streamlit dashboard, <code>streamlit-agraph</code> viz)
Phase 5	Deliverables & compliance trace	<code>trace_logger.py</code> , validation scenarios, this report, presentation decks

3 Source-File Inventory

File	Lines	Responsibility
<code>app.py</code>	~960	Streamlit dashboard, three-pane layout, graph viz, KPI metrics
<code>cognitive_engine.py</code>	~350	Track 3 Graph-RAG: extraction, retrieval, synthesis
<code>data_pipeline.py</code>	~280	CSV aggregation, PII synthesis, anonymisation, bucketisation, note generation
<code>ingest.py</code>	~660	Neo4j ingestion, NLP enrichment, PDF triple extraction, normalisation
<code>rule_engine.py</code>	~260	Track 1 rule definitions and evaluation
<code>similarity_engine.py</code>	~330	Track 2 learned similarity scoring
<code>trace_logger.py</code>	~40	JSON event logging
<code>generate_ppts.py</code>	~750	Reproducible PPT deck generation with matplotlib charts
<code>start.bat</code>	~145	Windows one-click launcher with GPU detection

<code>docker-compose.yml</code>	~70	Service definitions and health checks
<code>Dockerfile</code>	~25	App container image

4 Testing Approach

The project applied a **layered testing approach** consistent with an MVP scope:

- 11. **Configuration smoke test** — `docker compose up -d --build` succeeds, all three services reach **healthy** status, the app responds to `GET /_stcore/health` with **200**.
- 12. **Pipeline integrity test** — Run `data_pipeline.run_pipeline()` against the UCI dataset; assert (a) row count matches expected, (b) all required columns are present, (c) no PII column survives anonymisation, (d) every row has a non-empty **description**.
- 13. **Ingestion verification** — Run `ingest.run_ingestion()`; verify Neo4j populates the expected node labels and relationship types via Cypher count queries (e.g. `MATCH (p:Patient) RETURN count(p)`).
- 14. **Track 1 unit testing** — For each rule, supply a parameter dict that triggers it and one that does not; assert the alert is emitted with the expected severity.
- 15. **Track 2 integration test** — Issue a known-risk patient query and assert (a) at least 3 candidates are returned, (b) the top-1 candidate has **diagnosis_code > 0** more often than chance over the corpus.
- 16. **Track 3 integration test** — Issue the same query and assert that the rationale contains *at least one* token corresponding to a retrieved graph node, validating that the LLM is grounding rather than hallucinating.
- 17. **End-to-end validation scenarios** — Three scripted patient cases (see §8.2) exercise the entire pipeline.

Note on test automation maturity. The MVP relies on manual / scripted execution of the above checks rather than a full pytest suite; formal test automation is identified as a Phase 6 deliverable in the roadmap (Appendix D, Recommendations).

Project Performance & Validation

1 Latency Benchmarks

End-to-end query latency was measured on two reference configurations:

Stage	CPU only	NVIDIA GPU
LLM + regex extraction	~ 2.0 s	~ 1.2 s
Track 1 rule evaluation	< 0.1 s	< 0.1 s
Track 2 graph similarity	~ 0.7 s	~ 0.6 s
Track 3 rationale synthesis	~ 7.5 s	~ 3.0 s
Total	~ 10.25 s	~ 4.85 s

Both configurations satisfy the < **15-second** primary objective (O3). The GPU configuration achieves **sub-5-second** end-to-end latency. A graphical breakdown is available in `ICDSS\_System\_Design\_Deck.pptx`, slide 8.

2 Validation Scenarios

Three scripted patient scenarios were used to verify end-to-end behaviour. Each scenario specifies an input, the expected Track 1 alerts, an expected qualitative Track 2 retrieval profile, and the expected Track 3 grounding behaviour.

Scenario A — Elderly Multi-Risk Cardiovascular Profile

- **Input:** "70-year-old male, BP 152/95, LDL 168 mg/dL, BMI 31, active smoker, family history of CVD."
- **Expected Track 1 alerts:** HTN_STAGE2, SMOKING_HTN_COMBO, ELDERLY_MULTI_RISK, SMOKING_CVD_RISK. Severity: 4 critical / 0 warning.
- **Expected Track 2 behaviour:** Top-3 cases dominated by elderly diseased patients with overlapping risk factors. Top similarity ≥ 0.55 .
- **Expected Track 3 behaviour:** Rationale references Hypertension Stage 2 threshold from the ACC/AHA guideline node and explicitly cites the rule alerts as input evidence.
- **MVP outcome:** All three tracks produce the expected qualitative profile.

Scenario B — Female with Atypical Angina

- **Input:** "55-year-old woman with intermittent chest pain, BP 138/86, cholesterol 245 mg/dL."
- **Expected Track 1 alerts:** HTN_STAGE1, HIGH_CHOL, CHEST_PAIN_EVAL, FEMALE_ATYPICAL. Severity: 0 critical / 4 warning.
- **Expected Track 2 behaviour:** At least one female matched case in the top-3.
- **Expected Track 3 behaviour:** Rationale acknowledges the elevated risk profile in women presenting with atypical angina (per ACC/AHA narrative).
- **MVP outcome:** All three tracks produce the expected qualitative profile.

Scenario C — Acute-Concern Syncope

- **Input:** "62-year-old man, recent fainting episode, exertional chest pain, BP 145/92."
- **Expected Track 1 alerts:** SYNCOPE_CARDIAC, HTN_STAGE2, CHEST_PAIN_EVAL. Severity: 3 critical / 0 warning.
- **Expected Track 2 behaviour:** Diseased candidates dominate the top-3.
- **Expected Track 3 behaviour:** Rationale highlights the criticality of ruling out cardiac syncope and recommends ECG, echocardiogram, Holter monitoring.
- **MVP outcome:** All three tracks produce the expected qualitative profile.

3 MVP Signal-Quality Metrics

The dashboard surfaces real-time *signal-quality* verdicts per track:

Track	Verdict logic (<code>build_track_metrics</code> , <code>app.py</code>)
-------	--

Track 1	Good if ≥ 3 structured params extracted and rule engine produced an interpretable safety signal.
Track 2	Good if ≥ 3 historical cases returned and top hybrid similarity ≥ 0.35 .
Track 3	Good if graph evidence ≥ 3 and rationale generated.
Overall	Composite 0–6 score; ≥ 5 = Good, ≥ 3 = Moderate, otherwise Needs Review.

These verdicts are not clinical accuracy claims; they are honest, real-time indicators of *whether the system has enough evidence to be useful for this query*.

4 Privacy Validation

The privacy posture (Objective O5) was validated by:

18. Running the full pipeline against the augmented UCI dataset.
19. Inspecting `processed_data/patient_master.csv` for the absence of `synthetic_name`, `synthetic_dob`, and `synthetic_patient_id` columns.
20. Inspecting `processed_data/trace.json` for the absence of any raw PII string.
21. Confirming that the `Anonymization` event in the trace contains only a hashed-ID prefix and the boolean flag `pii_removed: true`.

All four checks pass in the current MVP build.

5 Comparison to Objective Targets

Objective	Target	Observed	Status
O1 — Three reasoning tracks	3	3	Met
O2 — Evidence trail per output	100 %	100 %	Met
O3 — End-to-end latency	< 15 s	4.85 s GPU / 10.25 s CPU	Met (with margin)
O4 — Manual steps after launch	0	0	Met
O5 — Raw PII persistence	0	0	Met

Project Conclusions — Findings & Recommendations

1 Key Findings

Finding 1 — Hybrid reasoning is materially more defensible than monolithic prediction. Across all three validation scenarios, the *combination* of a deterministic rule alert, a retrieved similar case, and a graph-grounded rationale gave a clinician reviewer substantially more material to work with than any single track in isolation. The act of presenting three independent outputs side-by-side is itself an explainability mechanism.

Finding 2 — Learning the right thing matters more than learning the most things. Track 2's deliberate scoping of "learning" to *retrieval weights* — rather than a full classifier — preserved full explainability while still adapting term importance to the actual clinical corpus. This is a useful pattern beyond CDSS: many "AI" applications can be re-cast as *learned retrieval scorers* with a substantial gain in interpretability.

Finding 3 — Local LLMs are operationally viable for clinical reasoning. Llama 3 8B running on commodity hardware (CPU or single consumer GPU) was sufficient for both extraction and rationale synthesis, with end-to-end latency well within the project target. This decisively addresses the data-residency objection that has historically blocked LLM-based CDSS.

Finding 4 — Constraint prompting is necessary but not sufficient. Strictly scoping the rationale-synthesis prompt to the retrieved graph evidence reduced hallucination significantly, but did not eliminate the variability inherent to LLM outputs. This is why the system surfaces *graph evidence count* as a first-class metric: a clinician should distrust a confident-sounding rationale built on weak grounding.

Finding 5 — Trace by construction is cheaper than trace as a feature. Recording the assembled prompt, the Cypher queries, and the rule firings into `trace.json` from the very first call cost almost nothing in development time and produced a complete audit trail "for free". Doing this retro-actively would have been substantially harder.

2 Limitations Honestly Stated

The project deliberately distinguishes between *design choices* and *limitations to be addressed*:

#	Limitation	Severity	Mitigation Path
L1	Synthetic clinical notes are generated from structured fields, not authentic narratives.	Medium	Replace with real-clinical-note corpus once an IRB-approved dataset is available.
L2	LLM outputs are non-deterministic and may vary across runs.	Medium	Lower temperature for rationale; build golden-output regression tests.
L3	Jaccard component treats unmatched terms equally outside learned weights.	Low	Future: add severity-weighted edge attributes.

3 Recommendations

My primary recommendations for a follow-on phase are:

22. **Build a golden test set.** Curate 20–50 patient vignettes with reviewed expected outputs for all three tracks, and run the full system against the set on every code change.
 23. **Introduce optional vector retrieval beside the graph.** A small embedding model (e.g. `all-MiniLM-L6-v2` running locally) would let Track 3 retrieve semantically-relevant guideline passages even when the canonical normalisation map misses an entity.
-

Appendix A — Project Proposal

A.1 Introduction

Cardiovascular diseases (CVDs) remain the leading cause of mortality globally. According to the World Health Organization (WHO, 2021), approximately 17.9 million deaths are attributed to CVDs annually, representing 32% of all global fatalities. In clinical settings, practitioners must not only process massive streams of real-time biometric data but also adhere to intricate clinical protocols for decision-making. This project proposes the development of ICDSS, which leverages a hybrid reasoning architecture—combining data-driven discovery with formal domain expertise—to deliver highly interpretable decision support. The core value of the system lies in transitioning from traditional "Black Box" models to a transparent reasoning chain, thereby augmenting the accuracy and safety of clinical interventions.

A.2 Background

With the advancement of digital healthcare, CDSS has emerged as a pivotal component of modern medical infrastructure. According to a Grand View Research (2024) industry report, the global CDSS market is projected to reach USD 10.7 billion by 2030, with a CAGR exceeding 10%. This robust growth is primarily driven by healthcare organizations' commitment to mitigating diagnostic errors and optimizing resource allocation.

However, existing systems face significant hurdles in real-world deployment. The McKinsey Global Institute has highlighted that while machine learning has the potential to generate substantial economic value within the healthcare sector, many algorithms fail to garner clinician trust due to a lack of interpretability and standardized frameworks. Consequently, engineering a system capable of demonstrating a "traceable reasoning process" rather than a mere "prediction" is essential for the successful commercialization of AI in medicine.

A.3 Literature Review

In high-stakes domains such as medicine, transparency and regulatory compliance are paramount. The National Institutes of Health (NIH, 2023) asserts that simultaneous improvements in explainability and predictive accuracy are required to accelerate the adoption of clinical AI. This project addresses two primary obstacles through its hybrid reasoning approach:

1. **Compliance risk:** Under the regulatory frameworks of Singapore PDPA and Global GDPR, medical system must be able to process sensitive data and explain self-automation process.
2. **Knowledge Fragmentation:** Most clinical experience lies in unstructured data such as professional guidelines and research papers. Traditional statistical model cannot process such symbolic knowledge directly. The integrated Cognitive System in this project can bridge this gap.

A.4 Project Scope & Reasoning Tracks

This project will integrate the three core intelligent reasoning technologies below in order to achieve multidimensional decision-making support in CVDs diagnosis:

1. **Decision Automation:** Integrate the standard clinical guidelines from the American Heart Association (AHA) and the European Society of Cardiology (ESC), and implement deterministic risk stratification via a rule engine. This approach ensures the system maintains stability and rigor when handling medical red-line scenarios.
2. **Knowledge Discovery:** By applying graph mining algorithms (such as the Jaccard similarity) to the UCI Heart Disease Dataset (UCI Machine Learning Repository, 1988), the system can retrieve digital twin historical cases that highly match the current patient's features, providing physicians with evidence-based reference benchmarks.
3. **Cognitive Systems:** By constructing a Neo4j knowledge graph and integrating large language models, the system supports interaction in natural language. This human-machine collaboration mode not only lowers the operational threshold of the system, but also enables visual traceability of diagnostic recommendations through Graph-RAG technology.

A.5 Data Strategy & Reliability

The dataset adopted in this project is sourced from the UCI Machine Learning Repository (1988), which contains real medical records from four international medical centers: Cleveland, Hungary, Switzerland, and Long Beach.

To address privacy challenges in data acquisition, the ICDSS system follows the Privacy-by-Design principle. Before data enters the inference engine, all Personally Identifiable Information (PII) undergoes hashing and categorical transformation. Furthermore, the system leverages NLP to extract non-quantitative descriptive fields from CSV files, converting fragmented medical records into entity-relationship structures within the knowledge graph. This approach significantly improves data density and overall quality.

A.6 System Design Philosophy

The architectural design of this system follows a logic-first principle and adopts Docker-based microservice deployment to ensure independent operation within hospital intranet environments. Its technical core lies in hybrid reasoning: the rule engine guarantees absolute compliance, graph mining provides empirical references, and the cognitive layer delivers semantic interpretation. This three-in-one design not only meets complex reasoning requirements for academic research but also features strong commercial deployment potential in engineering practice.

Appendix B — Mapping to Modular Courses (MR · RS · CGS)

The Practice Module requires that the deliverable demonstrate the application of techniques and skills from each of the three modular courses. The table below maps every implemented system functionality to the originating course module(s) and to the specific concepts taught.

B.1 MR — Machine Reasoning

System functionality	Concept(s) applied	Source file(s)
Deterministic rule engine	Forward-chaining rules; condition–action production rules	<code>rule_engine.py</code>
Severity-ranked alert prioritization	Conflict resolution / agenda strategies	<code>rule_engine.evaluate_rules</code> (severity sort)
Clinical safety guard-rails	Knowledge-engineering best practice (rules from authoritative source)	<code>rule_engine.RULES</code> (ACC/AHA + WHO PEN derived)
Rule-base introspectability	Whitebox knowledge representation	<code>rule_engine.RULES</code> (every rule has <code>id</code> , <code>name</code> , <code>severity</code> , <code>message</code> , <code>source</code>)
Rule trace logging	Knowledge-system audit	<code>trace_logger.log_event("RuleEvaluation", ...)</code>
Hybrid neural-symbolic integration	Symbolic outputs injected into LLM prompt	<code>cognitive_engine.synthesize_rationale</code> (rule alerts in prompt)

B.2 RS — Reasoning Systems

System functionality	Concept(s) applied	Source file(s)
Knowledge graph as reasoning substrate	Graph-based knowledge representation	Neo4j 5.26 schema (§6.2)
Graph projection and	Cypher pattern matching	<code>cognitive_engine.retrieve_graph_context</code> , <code>similarity_engine._hybrid_similarity_cypher</code>

traversal		
Jaccard set similarity	Similarity-based reasoning; <code>gds.nodeSimilarity</code> family	<code>similarity_engine._weighted_term_similarity</code>
Learned weighting of evidence	Evidence-weighted reasoning; lift / Bayesian-flavoured priors	<code>similarity_engine._learn_term_weights</code> , <code>_learn_numeric_weights</code>
Hybrid retrieval scoring	Multi-criteria reasoning combination	<code>similarity_engine._hybrid_similarity_cypher</code> ($0.55 \cdot \text{term} + 0.35 \cdot \text{numeric} + 0.08 \cdot \text{boost}$)
Case-based reasoning ("digital twins")	CBR retrieval / adaptation	Track 2 top-K with score decomposition
Conflict-of-evidence handling	Side-by-side multi-source presentation	<code>app.py</code> three-pane layout
Reasoning trace explainability	Explainable AI via trace + score decomposition	<code>trace_logger</code> , <code>app.build_track_metrics</code>

B.3 CGS — Cognitive Systems

System functionality	Concept(s) applied	Source file(s)
Local LLM serving (Llama 3 8B)	Foundation models; cognitive architectures	<code>docker-compose.yml</code> (Ollama service)
Symptom and parameter extraction	LLM-based information extraction; constrained JSON output	<code>cognitive_engine.extract_symptoms_from_text</code> , <code>extract_patient_params</code>
Free-text NLP enrichment of patient notes	LLM-augmented IE on synthetic clinical narratives	<code>ingest.nlp_enrich_patients</code>
PDF triple extraction	LLM-based knowledge extraction; chunking strategy	<code>ingest.extract_triples_from_chunk</code>
Graph-Augmented RAG	RAG architecture; constrained generation grounded in retrieved evidence	<code>cognitive_engine.run_cognitive_pipeline</code>
Prompt engineering with explicit grounding	Prompt design; hallucination mitigation	<code>cognitive_engine.synthesize_rationale</code> (explicit "do not invent" instruction)
LLM + deterministic dual-path extraction	Robustness pattern; cognitive architecture redundancy	<code>cognitive_engine._fallback_extract_*</code>
Synonym normalisation across sources	Lexical / cognitive entity reconciliation	<code>ingest.NORMALIZATION_MAP</code>

Appendix C — Installation and User Guide

C.1 System Requirements

Resource	Minimum	Recommended
OS	Windows 10 with WSL2	Windows 11 / macOS / Linux
RAM	16 GB total (16 GB allocated to Docker if using <code>llama3:8b</code>)	32 GB
Disk	25 GB free for images, models, data	50 GB
GPU	Not required (CPU mode)	NVIDIA GPU with ≥ 6 GB VRAM
Network	Required only for first-time image and model pulls	—

C.2 Prerequisites

24. **Docker Desktop** with the WSL2 backend enabled. In Settings → Resources, allocate at least **16 GB RAM** and **4 CPU cores**.
25. **Git** (optional, for cloning the repository).
26. **NVIDIA driver and `nvidia-smi`** if GPU acceleration is desired (auto-detected by `start.bat`).

C.3 Acquiring Source Data

Place the following into `./raw_data/` before running the pipeline:

27. **UCI Heart Disease CSV** — download from <https://archive.ics.uci.edu/dataset/45/heart+disease> (or the consolidated Kaggle mirror) and save as `heart_disease_uci.csv`.
28. **Clinical guideline PDF (at least one)** — recommended:
 - 2017 ACC/AHA Hypertension Guideline — <https://www.jacc.org/doi/10.1016/j.jacc.2017.11.006>
 - WHO PEN — [https://www.who.int/publications/i/item/who-package-of-essential-noncommunicable-\(pen\)-disease-interventions-for-primary-health-care](https://www.who.int/publications/i/item/who-package-of-essential-noncommunicable-(pen)-disease-interventions-for-primary-health-care)

C.4 Quickstart (Windows)

From a PowerShell or Command Prompt window inside the project folder:

```
start.bat
```

`start.bat` will:

29. Verify Docker is installed and running.
30. Create the bind-mounted folders (`raw_data`, `processed_data`, `data`) if missing.
31. Write a default `.env` file if missing.
32. Detect an NVIDIA GPU and write a Compose override accordingly (or fall back to CPU mode).
33. If GPU VRAM is below 6 GB, switch the `.env` to `llama3.2:1b` for one-click reliability.

34. Run `docker compose up -d --build`.
 35. Wait for the Streamlit health check at `http://127.0.0.1:8501/_stcore/health`.
 36. Open the dashboard in the default browser.
- To force regeneration of the Compose override:

```
start.bat --reconfigure
```

C.5 Quickstart (macOS / Linux)

```
docker compose up -d --build
open http://127.0.0.1:8501    # macOS
# or: xdg-open http://127.0.0.1:8501    # Linux
```

C.6 Using the Application

37. Open the Streamlit dashboard at `http://127.0.0.1:8501`.
38. In the sidebar, click **Run Data Pipeline**. This aggregates, anonymises, bucketises, and note-enriches the CSV(s) in `./raw_data/`.
39. Click **Ingest into Neo4j**. Optionally enable LLM note enrichment and PDF guideline extraction (these can take several minutes on first run).
40. Type a patient case in plain English in the chat input. Example:

"70-year-old male, BP 152/95, LDL 168 mg/dL, BMI 31, active smoker, family history of CVD."
41. Inspect the three-pane reasoning output (Track 1 alerts, Track 2 cases, Track 3 rationale), the MVP signal-quality strip, and the interactive knowledge graph below.

C.7 Stopping the System

```
docker compose down
```

To also remove the persisted graph and model cache:

```
docker compose down -v
rm -rf data/
```

Appendix D — Individual Project Report

Role on the project: Sole author and engineer. End-to-end ownership of architecture, data pipeline, all three reasoning tracks, dashboard, validation, presentation materials, and this report.

1. Personal Contribution to the Project

My contribution involved the architectural design and implementation of a three-service Docker environment—integrating **Streamlit**, **Neo4j**, and **Ollama (Llama 3)**. I engineered the system to be

production-ready, featuring health-check-gated service ordering and an automated Windows launcher that detects GPU VRAM to dynamically toggle between model sizes (Llama 3 8B vs. Llama 3.2 1B).

The technical core of my work was the creation of a **triple-track reasoning engine**:

- **Track 1 (Deterministic):** I curated and encoded twenty clinical safety rules from ACC/AHA guidelines into a rule engine that provides high-fidelity alerts.
- **Track 2 (Discovery):** I designed a hybrid similarity scorer. Instead of a black-box model, I built a system that learns feature weights from diagnosis lift and clinical priors, ensuring every "similar case" match is mathematically decomposable and explainable.
- **Track 3 (Cognitive):** I implemented a Graph-RAG pipeline that utilizes a Neo4j knowledge graph to ground the LLM. This included a dual-path extraction layer that combines LLM logic with regex fallbacks to ensure zero-hallucination rationale synthesis.

Beyond the code, I automated the entire delivery pipeline. I developed scripts to generate the project's promotional slide decks and this final report directly from version-controlled markdown, ensuring that the engineering logic and the project documentation remained perfectly synchronized.

2. The Most Useful Lessons Learned

The most profound lesson from this project is that **hybrid reasoning is materially more defensible than monolithic prediction**. Initially, I viewed "AI" as a single end-to-end model. However, building three distinct tracks—deterministic, similarity-based, and LLM-synthesized—taught me that safety-critical systems require a "defense-in-depth" approach. Each track compensates for the others' weaknesses: rules provide the floor, similarity provides context, and the LLM provides the narrative.

Secondly, I learned that **constrained generation** is what turns LLMs from a liability into an asset. By strictly scoping Llama 3 to retrieved graph evidence and explicitly forbidding it from "inventing" information, I achieved clinician-defensible outputs on commodity hardware. This disproved my assumption that serious LLM workloads require massive cloud clusters; with the right engineering discipline, local foundation models are operationally viable today.

Finally, I realized the value of **"Audit-by-Construction."** By implementing a trace-logger from day one—recording every Cypher query and assembled prompt—I built a system that is inherently transparent rather than one that requires retrofitted explainability.

3. Application in Other Situations and Workplaces

The patterns developed in ICDSS are highly transferable to my professional work, particularly in high-stakes enterprise environments:

- **Hybrid Decision Support:** I plan to apply the "Three-Track" recipe to **regulatory compliance triage** and **IT incident root-cause analysis**. By encoding "must-hold" compliance rules alongside a similarity engine for historical cases, I can provide users with a "signal-quality verdict" that significantly increases trust in automated recommendations.

- **Graph-RAG for Knowledge Management:** The method of grounding an LLM in a domain-specific knowledge graph is a far superior default for internal technical documentation assistants than simple "long-context" stuffing. I intend to adopt this "ground-and-verify" posture for any internal LLM deployment to ensure policy-accuracy.
- **Engineering Hygiene:** The "Everything as Code" philosophy—specifically treating reports and presentations as reproducible scripts—will be my new standard for recurring executive briefings. This eliminates manual rework and ensures that the data presented to stakeholders is always a direct reflection of the latest system state.

In conclusion, ICDSS has shifted my perspective from being an "AI developer" to a "**Systems Architect for Trust.**" The true product is not the model itself, but the verifiable evidence and the transparent reasoning process that supports the final decision.

Appendix E — References

E.1 Clinical Sources

42. Whelton, P. K., Carey, R. M., Aronow, W. S., et al. (2018). *2017 ACC/AHA/AAPA/ABC/ACPM/AGS/APhA/ASH/ASPC/NMA/PCNA Guideline for the Prevention, Detection, Evaluation, and Management of High Blood Pressure in Adults*. Journal of the American College of Cardiology, 71(19), e127–e248. <<https://doi.org/10.1016/j.jacc.2017.11.006>>
43. World Health Organization. (2020). *WHO Package of Essential Noncommunicable (PEN) Disease Interventions for Primary Health Care*. Geneva: WHO. <[https://www.who.int/publications/i/item/who-package-of-essential-noncommunicable-\(pen\)-disease-interventions-for-primary-health-care](https://www.who.int/publications/i/item/who-package-of-essential-noncommunicable-(pen)-disease-interventions-for-primary-health-care)>
44. World Health Organization. (2024). *Cardiovascular Diseases (CVDs) — Fact Sheet*. <[https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds))>

E.2 Datasets

45. Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1988). *Heart Disease Data Set*. UCI Machine Learning Repository. <<https://archive.ics.uci.edu/dataset/45/heart+disease>>
46. Karim Sony, R. *Heart Disease Data (Kaggle mirror, consolidated CSV)*. <<https://www.kaggle.com/datasets/redwankarimsony/heart-disease-data>>

E.3 Software, Frameworks, and Models

47. Streamlit. (2024). *Streamlit Documentation*. <<https://docs.streamlit.io/>>
48. Neo4j. (2024). *Neo4j 5 Operations and Cypher Manual*. <<https://neo4j.com/docs/>>
49. Neo4j. (2024). *Graph Data Science (GDS) Library Reference*. <<https://neo4j.com/docs/graph-data-science/current/>>
50. Ollama. (2024). *Ollama — Run large language models locally*. <<https://ollama.com/>>

51. Meta AI. (2024). *Llama 3: Open Foundation and Fine-Tuned Chat Models*.
<<https://llama.meta.com/llama3/>>
52. **pdfplumber** project. (2024). *pdfplumber — Plumb a PDF for detailed information about each char, rectangle, line, etc.* <<https://github.com/jsvine/pdfplumber>>
53. Faker contributors. (2024). *Faker — generate fake data for any purpose*. <<https://faker.readthedocs.io/>>
54. Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. *Computing in Science & Engineering*, 9(3), 90–95. <<https://doi.org/10.1109/MCSE.2007.55>>

E.4 Methods and Concepts

55. Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*.
<<https://arxiv.org/abs/2005.11401>>